CIS 4020 · DATA STRUCTURES

Advanced Trees: Overview and Objectives

Balanced search trees: 2-3 trees, 2-3-4 trees, and B-trees

1. Overview

A binary search tree is only as fast as it is short. Searching, inserting, and deleting all walk a single path from the root toward a leaf, so the work depends on the height of the tree. When items arrive in a friendly order, the tree stays bushy and the height grows like $\log n$. When they arrive in sorted or nearly sorted order, the tree leans to one side, the height grows like n, and the binary search tree performs no better than a linked list. Nothing in an ordinary binary search tree prevents that from happening.

This unit is about trees that refuse to let it happen. A **balanced search tree** reorganizes itself during insertion and deletion so that its height always stays logarithmic, no matter what order the items arrive in. We study three closely related members of this family.

- The **2-3 tree**, where a node holds one or two values and has two or three children, and every leaf sits on the same level.
- The **2-3-4 tree**, which allows one more value and one more child per node, and opens up a choice about *when* a full node is split.
- The **B-tree**, which generalizes both of these to nodes with up to m children. B-trees were designed for data stored on disk, and variants of them are still the standard index in relational databases and appear in many file systems today.

The big idea that ties the three together is the **split**. Instead of growing downward from the leaves like a binary search tree, these trees grow upward: when a node overflows, its middle value moves up into the parent, and the tree only gets taller when the root itself splits. That single habit is why every leaf always stays on the same level.

This unit builds directly on the trees unit. You will use binary search tree ordering, inorder traversal, and the inorder successor again here, and the algorithm analysis unit gives us the language to say exactly what balance buys us.

We will do demonstrations, including the general rules for splitting, redistributing, and merging nodes, followed by worked insertion and deletion sequences. When we look at code for these trees, we will look at it in both Java (CIS) and C/C++ (CS). The best preparation is a pencil. Take a short list of numbers, insert them one at a time into an empty 2-3 tree, and check your final tree with an inorder traversal. If the traversal is not in sorted order, something went wrong along the way.

2. Learning objectives

By the end of this unit you should be able to:

Remember and understand

- Explain why the efficiency of a binary search tree depends on its height, and how the order of insertions and deletions can make that height grow linearly.
- Define a balanced search tree, and name the common examples: 2-3 tree, 2-3-4 tree, B-tree, red-black tree, and AVL tree.
- Define a 2-node, a 3-node, and a 4-node by the number of values and children each has, and state the rules for a 2-3 tree and a 2-3-4 tree, including the rule that all leaves are on the same level.
- Describe how values are ordered inside a node and across its subtrees, and explain why a 2-3 tree is not a binary search tree even though it keeps the binary search tree ordering property.
- Describe the inorder traversal of a 2-3 tree (left, small, middle, large, right) and explain why it visits the values in sorted order.
- Explain what splitting, redistributing, and merging do to a node, and why splitting a full root is the only way the tree gets taller.
- Distinguish a preemptive (top-down) split from a non-preemptive (bottom-up) split in a 2-3-4 tree.
- Describe how deletion from a 2-3-4 tree turns each 2-node on the search path into a 3-node or 4-node so that the deletion finishes in one pass.
- Define a B-tree of order m, state its properties, and recognize the 2-3 tree and the 2-3-4 tree as B-trees of order 3 and order 4.
- Explain why B-trees were invented, and why they are a natural fit for data stored on disk and for database indexes.
- Describe the red-black tree, the AVL tree, the B+ tree, and the R-tree, and state what each is used for.

Apply

- Search a 2-3 tree or a 2-3-4 tree for a given value, naming the nodes visited.
- Insert a sequence of values into an initially empty 2-3 tree by hand, showing every leaf split, internal node split, and root split.
- Delete a value from a 2-3 tree by hand, including swapping an internal value with its inorder successor, redistributing with a sibling, merging with a sibling, and removing an empty root.
- Insert a sequence of values into a 2-3-4 tree using preemptive splits, and again using non-preemptive splits.
- Insert a sequence of values into a B-tree of a given order m.

- Perform an inorder traversal of a 2-3 tree or a 2-3-4 tree, and use it to check that a tree you built by hand is correct.
- Trace a Java (CIS) or C/C++ (CS) implementation of searching and inserting in a 2-3 tree or a 2-3-4 tree, showing which nodes are visited, where splits occur, and what the tree looks like after each call.
- Write or complete code for a balanced tree node that can act as a 2-node, 3-node, or 4-node, including the method that chooses which child to follow during a search.

Analyze

- Explain why insertion by splitting keeps every leaf on the same level, and why deleting an empty root is the only way the tree gets shorter.
- Determine the smallest and largest possible height of a 2-3 tree that holds n values, and use those bounds to explain why search, insertion, and deletion are $O(\log n)$.
- Trace how a single insertion can cause a chain of splits up to the root, and how a single deletion can cause a chain of merges up to the root.
- Insert the same sequence into a 2-3-4 tree with preemptive and with non-preemptive splitting, explain why the two final trees differ, and identify when a preemptive split happens even though it was not needed.
- Explain how increasing the order m of a B-tree reduces its height, and why fewer levels matter so much when every node visited costs a disk read.
- Compare 2-3 trees, 2-3-4 trees, and B-trees by node size, splitting rules, and height.
- Connect each part of a balanced tree implementation to the hand-drawn rules, identifying where the code splits a full node, moves a value up to the parent, redistributes or merges during deletion, and creates a new root.

Evaluate

- Judge whether a given tree is a valid 2-3 tree, 2-3-4 tree, or B-tree of order m, and identify every rule it breaks.
- Choose between a plain binary search tree and a balanced search tree for a described situation, such as data that arrives already sorted, and justify the choice.
- Critique claims such as "a 2-3 tree is just a binary search tree with bigger nodes" or "preemptive splitting always produces a shorter tree," using the rules and examples from this unit.
- Weigh the trade-off of larger nodes in a B-tree: fewer levels to read against more values to examine and more unused space inside each node.
- Judge whether a given balanced tree implementation is correct, and identify errors such as children lost or misassigned during a split, values left out of order inside a node, or a root split that never increases the height.